

定位器指南

定位器 (locator) 决定了案例找不找得到元素。****案例不稳定，八成问题出在定位器上。****

本篇讲的是「怎么选、怎么写」。强制性的规范条文和违规判定标准在《XPath 編集規約》里，两篇配合看。

五种定位方式怎么选

优先级顺序

ATP 支持五种 `locator\_type`，按推荐程度排序：

类型	推荐度	适用场景
`ID`		元素有稳定的 id 属性
`CSS`		结构简单、能用属性选择器搞定
`XPATH`		需要按文本查找、或需要向上找父级
`NAME`		表单元素
`LINK_TEXT`		纯文本链接

****能用 `ID` 就别用别的**** —— 它最快、最短、最不容易受页面改版影响。

但前提是那个 id 得是稳定的。前端框架自动生成的 id (比如 `ext-gen1234`、`input-uid-98a7f`) 每次渲染都可能变，这种 id 比不用还糟糕，因为它会给人一种「我用了 id 所以很稳」的错觉。

CSS 与 XPath 的分工

两者能力有重叠，但各有不可替代的地方：

****只有 XPath 能做的：****

- 按文本内容查找元素 (`//button[text()='提交']`)
- 从子元素向上找父元素 (`//span[@class="badge"]/ancestor::tr`)
- 按位置取第 n 个兄弟节点

****CSS 更好的地方：****

- 语法短，可读性高
- 引擎执行更快 (差距在大型页面上才明显)
- 属性匹配写起来更自然 (`[data-testid="submit"]`)

实践上的建议：****默认用 CSS，遇到 CSS 表达不了的再上 XPath****。不要一上来就写 XPath，很多人这么做只是因为浏览器的「复制 XPath」按钮太顺手 —— 而那个按钮生成的恰恰是最糟糕的那种绝对路径。

属性选择的优先级

稳定性排序

选用哪个属性来定位，直接决定了案例能活多久。按稳定性从高到低：

1. `**`data-testid`**` —— 专门为测试而加的属性。它的唯一用途就是被测试代码引用，所以前端改版时不会动它。**这是最理想的选择**
2. `**`name`**` —— 表单字段的名称，和后端接口的参数名绑定，改动成本高，因此比较稳定
3. `**`id`**` —— 前提是人工命名的，不是框架生成的
4. `**`class`**` —— 不稳定。class 是给样式用的，改个配色就可能变
5. **文本内容** —— 最不稳定。文案随时会改，多语言站点还会随语言变

如果被测系统还没有 ``data-testid``，**推动前端加上它是性价比最高的投入**。这件事应该在项目早期就提，越晚补代价越大。

不要依赖 class

用 class 定位是新人最常见的坏习惯，因为浏览器 DevTools 里 class 看起来最显眼。

问题在于 class 的存在意义是控制样式，它随时会因为纯视觉的改动而变化 ——

换个按钮颜色、调整整个间距、升级一次 UI 组件库，都可能让 ``btn-primary`` 变成 ``btn-main`` 或者多出一个 ``btn-lg``。

如果不得不用 class，**用属性包含匹配而不是完全相等**：

```
...  
CSS: [class*="submit-btn"]  
XPath: //button[contains(@class,"submit-btn")]  
...
```

这样至少能扛住 class 列表里增删了别的类名的情况。

常见错误

绝对路径

```
...  
/html/body/div[2]/div[1]/form/div[3]/input  
...
```

浏览器「复制完整 XPath」生成的就是这种。它记录的是从根节点数下来的每一层位置，**任何一层多一个 div 就全断了** —— 而前端加个包裹层是再常见不过的改动。

这是规范明令禁止的第一条，违反会被校验器判为 ERROR。改法是找一个稳定的锚点从中间切入：

```
...  
//form[@name="login"]//input[@name="username"]  
...
```

依赖自动生成的 id

```
...  
//*[@id="ext-gen1234"]  
//div[@id="mat-input-7"]  
...
```

`ext-gen`、`mat-input`、`el-id` 这类前缀是组件库自动生成的，序号跟渲染顺序有关，页面上多一个组件就会全部错位。

这种定位器的恶劣之处在于**它今天能跑通**，所以写的时候不会发现问题，等到某次改版后一批案例集体挂掉，才发现是这个原因。

索引下标

```
...  
//table//tr[3]/td[2]  
...
```

按位置取第 3 行第 2 列，看起来很直观，但列表顺序会因为排序规则、数据变化、分页而改变。今天的第 3 行明天可能是第 5 行。

改成按内容定位：

```
...  
//tr[td[text()='ORD-20250115-001']]//td[@data-col="status"]  
...
```

这样即使行序变了也能找对。

过长的链式定位

```
...  
//div[@class="page"]/div[@class="content"]/div[@class="panel"]/div[@class="body"]/table/tbody/tr[1]/td[3]/span/a  
...
```

层级越多，断的概率越大。每一层都是一个可能变化的点。

用 `//` 跳过中间的无关层级，只保留有意义的锚点：

```
...
```

```
//div[@data-testid="order-panel"]//a[@class*="detail-link"]
```

...

自检清单

写完一个定位器，问自己三个问题：

1. ****前端加一层 div 会不会断？**** —— 会断说明用了绝对路径或过长的链式
2. ****这个属性值是人写的还是机器生成的？**** —— 机器生成的一律不能依赖
3. ****数据变了会不会失效？**** —— 用了索引下标就会

三个都过了，这个定位器大概率能活很久。

正式的违规判定标准和评审要求见《XPath 編集規約》与《案例评审 checklist》。